AI Agent for RTL Design and Optimization

Design document: architecture, phases, tools, files, guardrails and build plan

One-line summary. Give the agent a design specification, or existing RTL, plus an SDC timing-constraint file. It checks the design, measures real timing, area and power with open-source EDA tools, and improves the RTL in a loop until the targets are met. It never edits the SDC, and every change is saved so engineers can see exactly what improved.

Contents

- 1. Goal and core idea
- 2. Ways to start (input modes)
- 3. Architecture: five layers
- 4. Tools we will use
- 5. The run, phase by phase
- 6. Functions the LLM may call
- 7. When the loop stops
- 8. Guardrails
- 9. Sequential designs (flip-flops, memory)
- 10. How the SDC is handled
- 11. Snapshots, files and the comparison report
- 12. The configuration file
- 13. Code layout
- 14. Agent memory
- 15. Why not LangChain or LangGraph
- 16. Using Groq's free LLM: token limits
- 17. Build order
- 18. Testing plan
- 19. Honest limits
- 20. Glossary
- 21. How to explain it in an interview

1. Goal and core idea

We are building a practical, reusable tool, not something tied to one design. The agent can:

- Take a design requirement and generate the RTL, and explore several different architectures for the same function.
- Take existing RTL, understand it, run synthesis and static timing analysis (STA), find problems, and improve it.
- Keep iterating with no fixed number of rounds: after each round it evaluates the results against the targets and decides if another round is needed.
- Work with any design and any SDC, so other people can plug in their own design or workflow.

The core idea

> **The LLM is the designer. The EDA tools are the judges.** The LLM writes and edits Verilog. It is never allowed to claim a design is faster or smaller. Only the tools measure that. The LLM proposes, the tools measure, and the LLM reads the measurements and proposes again.

2. Ways to start (input modes)

You give the agent	What it does first
Spec only	Writes RTL, a testbench with a reference model, and a draft SDC from the spec. The user reviews the SDC and testbench because the spec is the only source of truth.
Existing RTL + SDC	Reads both, checks they match (do the clocks and ports in the SDC exist in the design?), then goes to synthesis and STA.
Existing RTL, no SDC	Asks for the target clock, builds a basic SDC, and continues.

After this first step all three modes use the same loop. Only the entry point differs, which is what keeps the tool general.

3. Architecture: five layers

Layer	Job
1. Controller	Python program that runs the loop and decides when to stop.
2. LLM brain	Reads compact reports, diagnoses problems, writes new RTL. Provider-agnostic (Groq now; can be swapped).
3. Tool adapters	Small wrappers around simulation, synthesis, STA and equivalence checks. Each is 'files in, numbers out'.
4. Guardrails	Rules enforced by code: SDC is read-only, tests must pass, best correct version is always kept.
5. History	One snapshot folder per round: RTL, diff, metrics, raw reports, change note.

Data flow: user config (spec or RTL, SDC, targets) goes into the controller. The controller calls the adapters to measure, sends a compact summary to the LLM, receives new RTL, runs the guardrail checks, saves a snapshot, and repeats. The output is the best RTL plus a comparison report.

4. Tools we will use

Agent side

Tool	Why
Python 3	Glue language; easy to read and explain.
Groq API (OpenAI-compatible)	The LLM brain on the free tier. Wrapped in one adapter so the provider can change later.
PyYAML + Pydantic	Read the config file and validate it.
difflib (built into Python)	Produce the RTL diff between rounds.
Jinja2	Fill in the HTML report template.
matplotlib or Plotly	Charts of slack, area and power across rounds.
pytest	Test our own tool code.
Git (optional)	Extra history for engineers who like it.

Chip-design side (open source)

Tool	Job
Yosys	Parses Verilog, reports hierarchy, ports, flip-flops; synthesizes to gates; reports area and cell counts.
OpenSTA	Reads the netlist, the cell library and the SDC; reports setup and hold slack, worst paths, estimated power.
Icarus Verilog or Verilator	Simulation to check functionality.
SymbiYosys / Yosys equivalence commands	Formal checks where they can prove 'same behavior'.
Standard-cell library (Nangate45 or Sky130, Liberty .lib)	The real gates with their delay, area and power.

Install note. The OSS CAD Suite bundle should cover Yosys, Icarus, Verilator and SymbiYosys, but OpenSTA may need a separate install (built from source, or obtained through OpenROAD). Verify this during setup.

5. The run, phase by phase

Phase	Steps
A. Setup	Read and validate the config. Create a run folder. Check every tool is installed; if not, stop with a clear message.
B. Starting design	Spec mode: LLM writes RTL, testbench, reference model and draft SDC. RTL mode: read the files; use the user's testbench, or use the original RTL as the golden reference.
C. Understand the design	Yosys reports modules, ports, flip-flop count, clock and reset signals. Check the SDC against the design (clocks and ports exist? unconstrained paths?). Build a short summary for the LLM.
D. Baseline (round 0)	Simulate, synthesize, run STA, save everything. This is the reference point for all later rounds.
E. Architecture exploration	LLM proposes several different architectures (e.g. small and slow vs big and fast). Each goes through simulate, synthesize, STA. The best candidates continue.
F. Improvement loop	(1) Diagnose from the worst paths, area and power. (2) Propose new RTL plus a change note stating the expected effect. (3) Compile, simulate against the testbench or golden reference, run formal checks if possible; reject on failure. (4) Synthesize and run STA with the unchanged SDC. (5) Compare with previous best and baseline. (6) Keep or reject, but always save. (7) Check the stop rules.
G. Finish	Write the final report. Place the best correct version in the output folder.

6. Functions the LLM may call

Only these exist. There is no general 'run any command' tool, which is a big part of why the system is safe. With Groq's token limits we run the measuring tools from the controller and send one compact message per round (see section 16), but the same functions define what the agent can do.

Function	What it does
read_file	Look at the RTL, SDC or a report.
write_rtl	Save a new design version (only inside the run folder).
run_simulation	Compile and run tests; return pass/fail and error text.
run_synthesis	Return gate count, area and flip-flop count.
run_sta	Return setup and hold slack, worst paths and power.
check_equivalence	Compare new behavior with the original.
get_history	See what earlier rounds tried and how they scored.

7. When the loop stops

- **Success:** all targets are met (setup and hold slack >= 0; area and power under their limits if set).
- **Plateau:** the last N rounds improved the score by less than a small threshold.
- **Stuck:** the same idea keeps being proposed, or several rounds in a row are rejected.
- **Budget:** a high safety cap on rounds, time or API usage, so it can never loop forever. This is a safety net, not the normal way a run ends.

The 'score' is defined by the user in the config, for example: timing must be met first, then minimize area, then power. The agent always keeps the best correct version found so far.

8. Guardrails

These are enforced by code, not by asking the LLM nicely.

- **The SDC is read-only.** The easiest way to 'fix' timing is to loosen the clock, which is cheating. The agent can change only the RTL and synthesis settings. If it thinks a constraint is unrealistic, it says so in the report but does not change it.
- **No version counts unless it passes correctness checks.**
- **The best correct version is always kept.** A bad round cannot destroy good work.
- **The LLM can write only inside the run folder.**
- **Latency policy.** The config says whether the number of clock cycles may change. If not, any change that shifts cycle timing is rejected.
- **Measurements come only from tools.** If the LLM says 'slack improved', the report shows the tool's number next to the claim.

9. Sequential designs (flip-flops, memory)

A flip-flop stores one bit and updates only on a clock tick. Groups of them make counters, state machines (FSMs), registers, FIFOs and small memories. Five things change compared with plain logic:

- **Timing is per clock cycle.** STA checks setup (signal arrives before the next tick) and hold (signal does not change too soon after a tick). The clock in the SDC is essential.
- **Testing needs a clock, a reset and many cycles**, compared every cycle against a reference model.
- **'Same behavior' is harder to define.** Adding a pipeline register makes a design faster but delays its answer by a cycle. The config's latency policy decides whether that is allowed. If allowed, outputs must match after a fixed delay and the report records the new latency.
- **New optimizations become possible:** pipelining, retiming, FSM state encoding, register sharing, clock gating, sync vs async reset.
- **New numbers are reported:** flip-flop count, achievable clock frequency, latency in cycles, hold violations.

Correctness: the ladder

Situation	How correctness is checked
User gives a testbench	Run it every round.
User gives RTL, no testbench	Original RTL is the golden reference. Same random inputs go through original and new versions; outputs are compared every cycle. Formal equivalence is added where the tools can prove it.
User gives only a spec	LLM writes RTL, testbench and reference model from the spec; the user reviews them.

Be careful with claims. Proving sequential equivalence formally is harder than for plain logic and the free tools only go so far. Random simulation against a reference is the main safety net, so the report should state how much behavior was actually covered. Hardware memory in scope: flip-flop based memories (register files, small FIFOs). SRAM macros are treated as black boxes and are out of scope.

10. How the SDC is handled

- An SDC file is the timing rulebook: clock speeds, input arrival and output required times, false paths, multicycle paths. The same Verilog can be 'fast enough' under one SDC and 'too slow' under another, so the SDC defines what 'good' means.
- **We do not write our own SDC parser.** SDC is a large language. OpenSTA already understands it, so we feed the file straight to it. The agent reads only simple things (clock names and periods) to help the LLM understand the design.
- **The agent checks the SDC for problems** before trusting any result: clocks that do not exist, ports with no timing, unconstrained paths. A missing constraint can make a design look better than it is.
- **Limit:** Yosys only partly understands SDC (mostly the clock period). Exceptions such as multicycle and false paths are fully respected only in the STA step. For stronger SDC-aware synthesis, a different flow can be plugged in through the adapter design.

11. Snapshots, files and the comparison report

Nothing is ever overwritten. Every round gets its own folder, including rejected rounds.

```
run_YYYYMMDD_HHMM/
  config.yaml
  round_00/                 (baseline)
    design.v                exact Verilog used in this round
    diff_vs_previous.patch  lines changed since the previous round
    metrics.json            slack, area, power, flip-flops, tests, equivalence
    reports/                raw Yosys and OpenSTA output, untouched
    change_note.md          what changed, why, what was expected
  round_01/ ...
  round_02/ ...
  best/                     final chosen design
  report.html               comparison table, charts, side-by-side diffs, summary
```

The report contains

- A round-by-round table. Each row shows the change vs the previous round and vs the original.
- Charts of slack, area and power across the rounds.
- For each round, a side-by-side RTL diff next to the metric changes.
- **Claim vs measurement:** the LLM's expectation next to what the tools measured, marked true or false.
- Rejected rounds with the reason, so engineers can learn from them and the agent avoids repeating them.
- A short plain-English summary, e.g. 'Timing was fixed in round 1 by adding a pipeline stage at a cost of 12% more area; round 2 recovered most of that area.'

Example table (made-up numbers, to show the shape only)

#	What changed	Slack	Area	Power	Tests	Verdict
0	Original design	-0.8 ns	1200	3.1 mW	Pass	Baseline
1	Pipeline register in multiplier	+0.1 ns	1350	3.4 mW	Pass	Kept: timing met
2	Shared two adders	+0.1 ns	1270	3.2 mW	Pass	Kept: area down
3	Merged registers back	-0.2 ns	1240	3.1 mW	Pass	Rejected: broke timing

Storage: plain folders first (anyone can open them). Git commits per round are an optional extra. Report format: HTML for the demo, with a lightweight Markdown copy.

12. The configuration file

Users describe their project in one YAML file. Nothing about a specific design is built into the tool.

Field	Meaning
mode	spec or rtl
spec text / RTL paths + top module	What to build or what to analyze
sdc path	The timing constraints (read-only for the agent)
testbench path (optional)	User's own tests
cell library path	Liberty .lib file
targets	Timing, area limit, power limit
priority order	How the score ranks timing, area, power
latency policy	Fixed, or allowed to change
budgets	Max rounds, time, API usage
llm provider and model	Groq model name and key (from environment, never stored in the file)
custom commands (optional)	Replace any tool step with the user's own flow

13. Code layout (one module per layer)

Module	Responsibility
config	Reads and validates the YAML.
adapters/	One file each for simulate, synth, sta, equiv. Same shape: files in, numbers out.
parsers	Turn tool output into clean numbers and short summaries.
llm	Provider-agnostic adapter: send messages, get a reply; handles retries and rate limits.
controller	The loop and the stop rules.
guardrails	Checks for SDC unchanged, tests passed, folder boundaries.
history	Snapshots, diffs, and the one-line list of tried ideas.
report	Builds the comparison report.
cli	The command people run.

14. Agent memory

The LLM itself has no memory. Every API call starts blank. Whatever the agent 'remembers' is text our code chooses to send again, so memory is a design decision.

Layer	Holds	Lives in	Lasts
Short-term (working)	Current task, latest RTL, latest results, last 2-3 rounds	The LLM's context window	One round; rebuilt each time
Run memory	Every round's RTL, diff, metrics, reports, notes; ideas tried and rejected	Files in the run folder	The run, and afterward
Project memory	Config, targets, SDC, testbench, preferences	Config files	As long as the project
Long-term, cross-run (optional)	General lessons, e.g. 'pipelining a multiplier fixed timing but cost area'	A small notes file or database	Across runs

Keeping short-term memory small

- Each round the LLM sees: baseline numbers, best version so far, the last two or three rounds in detail, a one-line list of every idea already tried with its result, and the targets and rules.
- Older reports and RTL versions are fetched only on request (get_history). This saves tokens and avoids confusing the model.

Cross-run learning is optional and risky: a trick that worked on one design may hurt another. If added, lessons are stored with their context, used only as hints, can be turned off, and measurements always win. The first version skips it; it is a clean 'future work' item. Do not confuse agent memory with the hardware memory (flip-flops) inside the designs. They are separate ideas that share a word.

15. Why not LangChain or LangGraph

They are good tools. We skip them for now for three reasons:

- **The goal is to explain every part.** Frameworks hide the loop inside their classes. A hand-written loop of about 100 lines you understand beats a framework you cannot see inside.
- **Our loop is simple:** call the LLM, run the tool, send the result back, check the stop rules, repeat. One model, a handful of small functions.
- **The important logic is ours anyway:** guardrails, stop rules and snapshots are custom code either way.

Costs of frameworks: they change often, debugging goes through extra layers, and every user of a 'reusable tool' must install them. What LangGraph would genuinely give: a visual graph, built-in save and resume, and a standard way to pause for human approval. Our loop already maps onto a graph, so it can be ported later. Plan: build by hand first, then optionally map it to LangGraph.

16. Using Groq's free LLM: token limits

Limits listed on Groq's rate-limit page on 9 Oct 2026 for the main text models (for example openai/gpt-oss-120b, openai/gpt-oss-20b, qwen/qwen3.8-27b). The page does not clearly separate free-tier from paid values, so check your own limits in the Groq console. Limits change often.

Limit	Value
Requests per minute	30
Requests per day	1,000
Tokens per minute	8,000
Tokens per day	200,000

Rough size of one call for a mid-size design (about 100-300 lines of RTL): instructions 1,000-1,500 tokens; design summary ~1,000; the RTL 1,500-4,000; summarized results 1,000-2,000; history ~1,000; the reply (new RTL) 1,500-4,000. Total about 7,000-13,000 tokens per call.

What this means

- The per-minute limit bites first. A large design in one request can be rejected outright.
- The daily limit gives roughly 5-15 rounds per day. Several calls per round multiplies the cost.
- Small designs (counter, FSM, small ALU) fit comfortably; large designs do not.
- Open models are generally less reliable at Verilog, so expect more rejected attempts, which also spend tokens.

Design changes to fit the limits

- **One LLM call per round.** The controller runs simulation, synthesis and STA itself, then sends one compact message with all results.
- **Send numbers, not raw reports.** Parsers reduce a huge timing report to the worst few paths and key metrics. Raw reports stay on disk.
- **Send only the relevant module**, and ask for a patch (changed lines) instead of the full file where possible.
- **History as one-liners**, e.g. 'Round 2: shared adders, area -6%, kept'.
- **Handle rate limits gracefully:** wait and retry, and save progress so a run can resume after the daily reset.
- **Count tokens before sending** and warn 'this design is too big for your current limit' instead of failing mysteriously.
- **Provider-agnostic LLM adapter:** Groq now, another provider later, without touching the controller.

17. Build order

Each step produces something that works and can be explained before the next layer is added.

Step	What gets built	What you learn
1	Install tools; run Yosys by hand on one Verilog file	What synthesis produces; where area comes from
2	Wrap it in a Python function; parse the area	Calling tools from code; reading reports
3	Add OpenSTA with an SDC; parse slack	Setup/hold slack; what the SDC controls
4	Add simulation	Testbench, clock, reset, pass/fail
5	Add history and snapshots (still no LLM)	Round folders, diffs, metrics files
6	Add the LLM adapter, then the proposal step	How the model is prompted and how replies are handled
7	Add the loop and stop rules	The agent loop itself
8	Add spec mode, architecture exploration, report	Spec to RTL; comparison reporting
9	Run on three very different designs without code changes	Proof that it is general

18. Testing plan

- **Unit tests** (pytest) for parsers, config validation, guardrails and stop rules, using saved sample tool outputs so tests run without the LLM.
- **Tool tests:** known small designs with known area and slack results.
- **Guardrail tests:** try to change the SDC, write outside the run folder, accept a failing design. All must be refused.
- **Generality test:** run the finished tool, unchanged, on three different designs, for example a small FSM, a design with a memory array, and an arithmetic block. If it works on all three, that is the evidence.
- **Resume test:** stop a run mid-way (simulating a rate limit) and confirm it resumes without losing the best version.

19. Honest limits

In scope: synthesizable digital RTL in Verilog (SystemVerilog as far as Yosys supports it), single or multiple clocks, any size the free tools handle.

- Out of scope: analog, code that cannot be synthesized, vendor macros and SRAM blocks (black boxes), very large SoCs.
- Power numbers come from default activity estimates and are rough unless real switching data is supplied.
- Open-source synthesis is good for learning and demos, not production signoff.
- Formal proof for sequential designs is limited; random simulation against a reference is the main safety net.
- Multi-clock-domain crossings get only limited checking.
- LLM-written RTL can be wrong, which is why it is never trusted without checks.
- The tool is only as good as its targets; vague targets give vague results.

The honest claim: **general-purpose for synthesizable digital RTL, with correctness checked against a user-supplied testbench or the original design.**

20. Glossary

Term	Plain meaning
RTL	Verilog code describing what the chip does, cycle by cycle.
Synthesis	Turning RTL into real logic gates from a cell library.
STA	Static timing analysis: checks whether signals arrive within a clock period.
SDC	File of timing rules: clocks, input/output delays, exceptions.
Slack	Spare time on a path. Positive means met; negative means too slow.
Setup / hold	Signal must arrive before the clock tick (setup) and not change too soon after it (hold).
Flip-flop	Stores one bit; updates on a clock tick.
FSM	Finite state machine: a design that moves between a fixed set of states.
Pipelining	Splitting a long path with registers to allow a faster clock, at the cost of extra cycles of latency.
Retiming	Moving registers across logic to balance path delays.
Testbench	A test that feeds inputs to a design and checks the outputs.
Golden reference	The trusted version a new version is compared against.
Equivalence checking	Proving or testing that two designs behave the same.
Liberty (.lib)	File describing each library gate's delay, area and power.
Token	A piece of text (about three-quarters of a word) that LLM limits are counted in.
Context window	The amount of text the LLM can see in one call.

21. How to explain it in an interview

> *"It's an LLM in a controlled loop. The LLM proposes RTL changes, but every decision comes from real synthesis and timing tools, every change must pass functional checks, the SDC can't be touched, and every round is saved so engineers can see exactly what improved and why."*

Questions to be ready for

- Why can't the LLM just say a design is faster? *Because only deterministic tools measure; the LLM only reads results.*
- How do you know the optimized RTL still works? *Simulation against a testbench or the original design every round, plus formal checks where possible; failing versions are rejected.*
- Why not just loosen the clock? *The SDC is read-only to the optimizer; unrealistic constraints are reported, not changed.*
- How does it know when to stop? *Targets met, plateau, stuck, or a safety budget; the best correct version is always kept.*
- What happens with a pipeline register that adds a cycle? *The latency policy in the config decides whether that is allowed.*
- Why no LangChain? *The loop is simple and I wanted full visibility; it maps onto LangGraph if needed.*
- What are the weaknesses? *Rough power numbers, limited sequential proof, free-model reliability and token limits, multi-clock checking.*